Secure Amazon RDS MySQL Access Using an EC2 Bastion Host (Jump Server)

Project Overview

This project demonstrates how to securely deploy and access an **Amazon RDS MySQL database** hosted inside a **private subnet** using an **Amazon EC2 Bastion Host (Jump Server)** deployed in a **public subnet** within a custom Amazon VPC.

The primary objective of this project is to implement a secure AWS networking architecture where the database is isolated from direct internet access while still allowing authorized administrators to manage it through a controlled access point.

This architecture closely resembles real-world production environments, where sensitive databases are never exposed to the public internet and are accessible only through secure intermediate systems.

Problem Statement

Databases contain critical business information and should never be publicly accessible.

If an Amazon RDS database is placed in a public subnet or exposed to the internet, it becomes vulnerable to unauthorized access, brute-force attacks, and security threats.

To overcome this issue, AWS recommends placing databases inside **private subnets** and using a **Bastion Host (Jump Server)** as the only secure entry point for administrators.

This project implements that recommended architecture.

Project Objectives

* Design a secure AWS networking architecture.
* Deploy an Amazon RDS MySQL database in a private subnet.
* Launch an EC2 Bastion Host in a public subnet.
* Configure networking components to enable secure communication.
* Restrict database access using Security Groups.
* Connect securely to the database using SSH.
* Execute SQL queries from the Bastion Host.
* Follow AWS networking and security best practices.

AWS Services Used

AWS Service	Purpose
Amazon VPC	Creates an isolated virtual network
Public Subnet	Hosts the Bastion Host
Private Subnet	Hosts the Amazon RDS MySQL database
Internet Gateway	Provides internet access to the public subnet
Route Tables	Controls traffic routing between subnets and the internet
Security Groups	Acts as a virtual firewall controlling inbound and outbound traffic

Network ACL	Additional subnet-level traffic filtering
Amazon EC2	Serves as the Bastion Host (Jump Server)
Amazon RDS MySQL	Managed relational database service
SSH	Secure remote connection to the Bastion Host
MySQL Client	Connects from the Bastion Host to the RDS database

---

Architecture Overview

The architecture consists of the following components:

```
Internet
    |
    ▼
EC2 Bastion Host (Public Subnet)
    |
    | SSH
    ▼
Amazon RDS MySQL (Private Subnet)
    |
    ▼
Execute SQL Queries
```

Traffic Flow:

```
Local Machine
    |
Internet
    |
Public IP
    |
EC2 Bastion Host
    |
Private Network
    |
Amazon RDS MySQL
```

The RDS instance never receives direct internet traffic.

Project Workflow

Step 1 – Create a Custom VPC

A custom Virtual Private Cloud was created to isolate all project resources.

The VPC provides:

* Private networking
* IP address management

* Resource isolation
* Secure communication

Example CIDR Block

```
10.0.0.0/16
```

Step 2 – Create Public and Private Subnets

Two separate subnets were created.

Public Subnet

Hosts:

* Bastion Host

Features:

* Public IP
* Internet access
* Internet Gateway route

Example:

```
10.0.1.0/24
```

Private Subnet

Hosts:

* Amazon RDS MySQL

Features:

* No Public IP
* No Internet access
* Accessible only from the Bastion Host

Example:

```
10.0.2.0/24
```

Step 3 – Create an Internet Gateway

An Internet Gateway was attached to the VPC.

Purpose:

* Enables internet connectivity for resources in the public subnet.

Step 4 – Configure Route Tables

Public Route Table

Routes:

```
10.0.0.0/16 → Local

0.0.0.0/0 → Internet Gateway
```

Associated with:

* Public Subnet

Private Route Table

Routes:

```
10.0.0.0/16 → Local
```

Associated with:

* Private Subnet

The private subnet has no direct internet route.

Step 5 – Configure Security Groups

Bastion Host Security Group

Inbound

```
SSH (22)

Source:

My Public IP
```

Outbound

```
All Traffic
```

Amazon RDS Security Group

Inbound

```
MySQL (3306)

Source:

Bastion Host Security Group
```

This configuration ensures that only the Bastion Host can access the database.

Step 6 – Launch the EC2 Bastion Host

An EC2 instance was launched in the public subnet.

Configuration:

* Amazon Linux
* t3.micro
* Public IPv4
* SSH Key Pair
* Bastion Host Security Group

The Bastion Host acts as the secure gateway to the private database.

Step 7 – Deploy Amazon RDS MySQL

An Amazon RDS MySQL instance was created.

Configuration:

* MySQL Engine
* Private Subnet
* No Public Access
* Dedicated RDS Security Group

The database is isolated from direct internet access.

Step 8 – Connect to the Bastion Host

From the local machine, SSH was used to connect to the EC2 instance.

Example:

```bash
ssh -i rds-bastion-key.pem ec2-user@<Public-IP>
```

After successful authentication, the administrator gains access to the Bastion Host.

Step 9 – Install MySQL Client

The MySQL client was installed on the EC2 instance.

Example:

```bash
sudo dnf install mariadb105
```

or

```bash
sudo yum install mysql
```

depending on the operating system.

Step 10 – Connect to Amazon RDS

The MySQL client was used to connect to the private RDS instance.

Example:

```bash
mysql -h <RDS-ENDPOINT> -u admin -p
```

The connection is established over the private VPC network.

Step 11 – Execute SQL Commands

Example:

```sql
SHOW DATABASES;

CREATE DATABASE portfolio_db;

USE portfolio_db;

CREATE TABLE employees(
```

```
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100),
department VARCHAR(50),
email VARCHAR(100)
);

INSERT INTO employees
(name,department,email)
VALUES
('Aryan','Cloud','aryan@example.com');

SELECT * FROM employees;
```

Security Best Practices Implemented

* Database deployed in a private subnet
* No public access enabled for RDS
* Bastion Host used as a secure jump server
* Principle of Least Privilege implemented using Security Groups
* Internet access limited to the Bastion Host
* Route Tables configured to isolate private resources
* Network ACLs configured for subnet-level protection
* SSH authentication secured using a Key Pair

Advantages of This Architecture

* Improved security
* Production-ready design
* No direct internet exposure for the database
* Controlled administrative access
* Scalable and reusable architecture
* AWS best-practice implementation
* Reduced attack surface

Real-World Use Cases

This architecture is commonly used in:

* Banking Applications
* Healthcare Systems
* Government Infrastructure
* Enterprise ERP Applications
* Financial Platforms
* Insurance Applications
* SaaS Platforms
* Internal Business Applications

Skills Demonstrated

* Amazon VPC
* Amazon EC2
* Amazon RDS
* AWS Networking
* Security Groups
* Network ACLs
* Internet Gateway
* Route Tables
* Linux Administration
* SSH
* MySQL
* Database Administration
* Cloud Security
* Infrastructure Design

Challenges Faced During Implementation

During implementation, several networking and connectivity issues were identified and resolved, including:

* RDS instance class availability in the selected AWS Region
* Security Group configuration for SSH and MySQL access
* Route Table and Internet Gateway association
* EC2 SSH connection timeout troubleshooting
* EC2 Instance Connect access verification
* Private database connectivity through the Bastion Host
* Creating and accessing databases after RDS deployment

These troubleshooting steps provided valuable practical experience in diagnosing and resolving common AWS networking issues.

Learning Outcomes

Through this project, I gained practical experience in:

* Designing secure cloud network architectures
* Deploying production-style AWS infrastructure
* Configuring secure communication between public and private resources
* Understanding VPC networking concepts
* Managing Amazon RDS databases
* Troubleshooting connectivity issues
* Implementing AWS security best practices
* Executing SQL operations on managed databases

Future Enhancements

Possible improvements to this project include:

* Implement Multi-AZ Amazon RDS for high availability
* Replace the Bastion Host with AWS Systems Manager Session Manager

* Store database credentials securely using AWS Secrets Manager
* Enable CloudWatch monitoring and alarms
* Configure automated backups and snapshot policies
* Implement IAM Database Authentication
* Automate infrastructure provisioning using Terraform or AWS CloudFormation
* Add an Application Load Balancer and Auto Scaling Group for application servers

---

Conclusion

This project demonstrates a secure, production-inspired AWS architecture for hosting a MySQL database in a private subnet and managing it through a Bastion Host. It showcases core cloud networking concepts, secure access patterns, and practical infrastructure management using AWS services. By implementing and troubleshooting this solution, I strengthened my understanding of VPC design, EC2 administration, Amazon RDS, Linux, SSH, and cloud security practices—skills that are directly applicable to real-world Cloud Engineering, DevOps, and Infrastructure roles.